

Get Hands-on with the IBM MQ AI Agents

Hands-on lab guide

Table of Contents

- 1 Introduction 5
 - 1.1 About this hands-on lab5
 - 1.2 Core Concepts5
 - 1.2.1 IBM MQ Console.....5
 - 1.2.2 IBM MQ Agents.....6
 - 1.2.3 Working with the agent9
 - 1.3 Conventions 10
- 2 Timing 10
- 3 Didn't get to complete the lab? 11
- 4 About this hands-on lab 11
 - 4.1 Agent capabilities 12
 - 4.2 Queue managers..... 12
 - 4.3 Lab environment reference 12
- 5 Part A – Introduction and hands-on examples..... 13
 - 5.1 Connecting to the AI Agents interface 13
 - 5.2 Hands-on example 1 – Exploring queue manager health 14
 - 5.3 Hands-on example 2 – Investigating channel status 15
 - 5.4 Hands-on example 3 – Correlating applications and queues 15
- 6 Part B — Scenario 1: Queue manager alert triage 16
 - 6.1 Option 1 – Support ticket (advanced) 16
 - 6.2 Option 2 – Guided prompts (assisted) 17
- 7 Part C — Scenario 2: Application performance triage 20
 - 7.1 Option 1 – Support ticket (advanced) 20
 - 7.2 Option 2 – Guided prompts (assisted) 21
- 8 Feedback section..... 23

9 Getting help and troubleshooting..... 23

10 Lab reference materials 24

Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed “as is” without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts.
In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply.”

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled, isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at: www.ibm.com/legal/copytrade.shtml.

1 Introduction

IBM MQ is a robust, reliable, and secure messaging solution. It simplifies and accelerates the integration of different applications across multiple platforms and supports a wide range of APIs, protocols, and languages. IBM MQ enables applications to communicate and exchange data in a reliable and scalable way, decoupling one application from another — smoothing out the problems of integrating applications running in different frameworks, languages, platforms, clouds, and locations.

Operating IBM MQ at scale means awareness of queue depths, channel states, application connections, and message flows across potentially dozens of queue managers. When something goes wrong — a queue filling up, an application hanging, a slow inter-QM channel — operators need to triage quickly and accurately to minimise the impact on business.

IBM MQ AI Agents bring the power of large language model (LLM)-based reasoning directly into the MQ operations workflow. Rather than manually issuing a sequence of MQSC commands and correlating the output, operators can ask natural language questions, and the agents will autonomously query the queue managers, interpret the results, and present findings in plain English. The agents can chain multiple queries together, identify patterns, and propose remediation steps — dramatically reducing mean time to resolution (MTTR).

In this lab you will use the IBM MQ AI Agents to triage two realistic operational scenarios, practising the kind of investigation that would typically require an experienced MQ to undertake a significant manual work.

1.1 About this hands-on lab

The objective of this lab is to get hands-on with the IBM MQ Agents by working with them to debug a live environment, undertake Root Cause Analysis (RCA) and determine remedial actions. You will interact with the AI agents using a chat interface provided by an IBM MQ Console instance that is accessed via a browser session on your student workstation. The IBM MQ console also provides *read* access to the queue managers so that you may use a combination of the agents and live configuration to reason about a problem.

1.2 Core Concepts

This lab refers to various components that have been preconfigured. The following sections introduce each of these and explain how they fit together.

1.2.1 IBM MQ Console

The IBM MQ Console (Figure 1) is a browser-based graphical user interface that lets you easily visualize and manage your IBM MQ systems. With the console you can quickly troubleshoot issues, inspect objects and much more. In this lab you have been granted *read* access only, so it will not be possible to change the configuration. This is because the queue manager instances that you are working with are shared with all students completing this lab. You can use the console to verify what the AI Agents are telling you and to build your understanding of the environment. The IBM MQ console is available at two endpoints (the instructor will tell you which one to use)

- <https://158.175.71.201:31414/ibmmq/console/>
- <https://158.177.74.129:31811/ibmmq/console/>

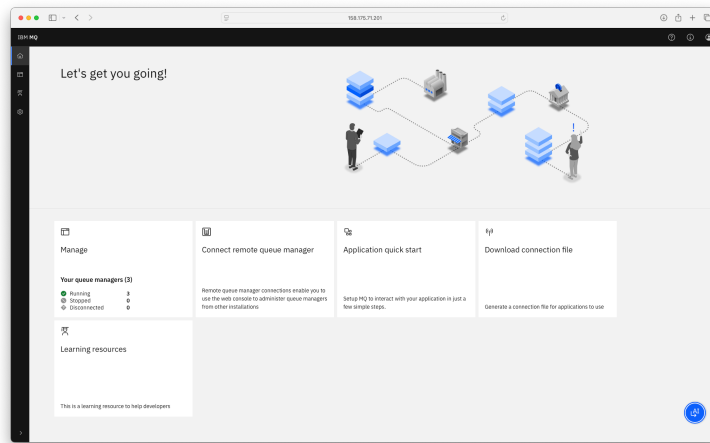


Figure 1, IBM MQ Console

1.2.2 IBM MQ Agents

The IBM MQ AI Agents are an AI-powered operations assistant for IBM MQ. The agents expose a conversational interface backed by an LLM. When you ask a question, the agents translate it into the appropriate MQ administrative operations (MQSC commands, REST API calls, PCF commands, Log analysis), execute them against the live queue managers in your environment, and summarise the results in natural language.

The agents have been pre-deployed to support this lab as shown in (Figure 2). You can learn more about the agent capabilities and deployment in the [IBM MQ documentation](#).

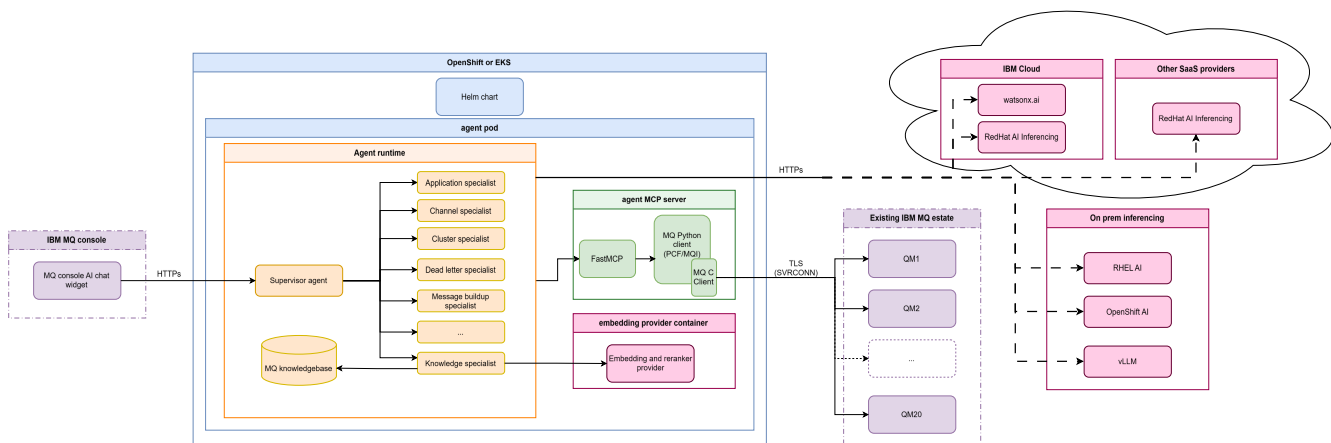


Figure 2, IBM MQ Agents Deployment

While the MQ agents we are working with are deployed to Red Hat® OpenShift® the queue managers they work with can be hosted in any supported IBM MQ environment or platform. For the purposes of this lab, the queue managers are deployed to virtual machines running Red Hat Enterprise Linux (RHEL).

The agent can use SaaS-based model hosting such as watsonx.ai, AWS Bedrock, or OpenAI API-based services such as Red Hat Inferencing. On-prem inferencing is also available using the OpenAI API, enabling options including RHEL AI, OpenShift AI, or vLLM.

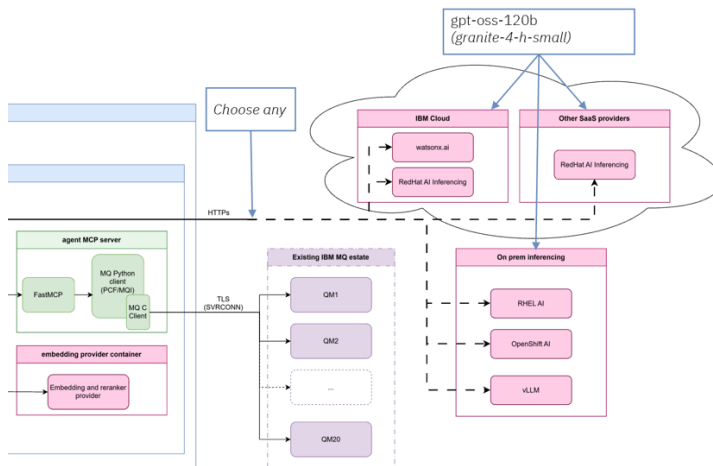


Figure 3, Runtime Inferencing Options

The latest 2.1 IBM MQ Agents release includes a new A2A capability enabling you to integrate the IBM MQ Agents with other compatible A2A agent deployments within your organisation. In addition, Inferencing Runtime support for AWS Bedrock has been added.

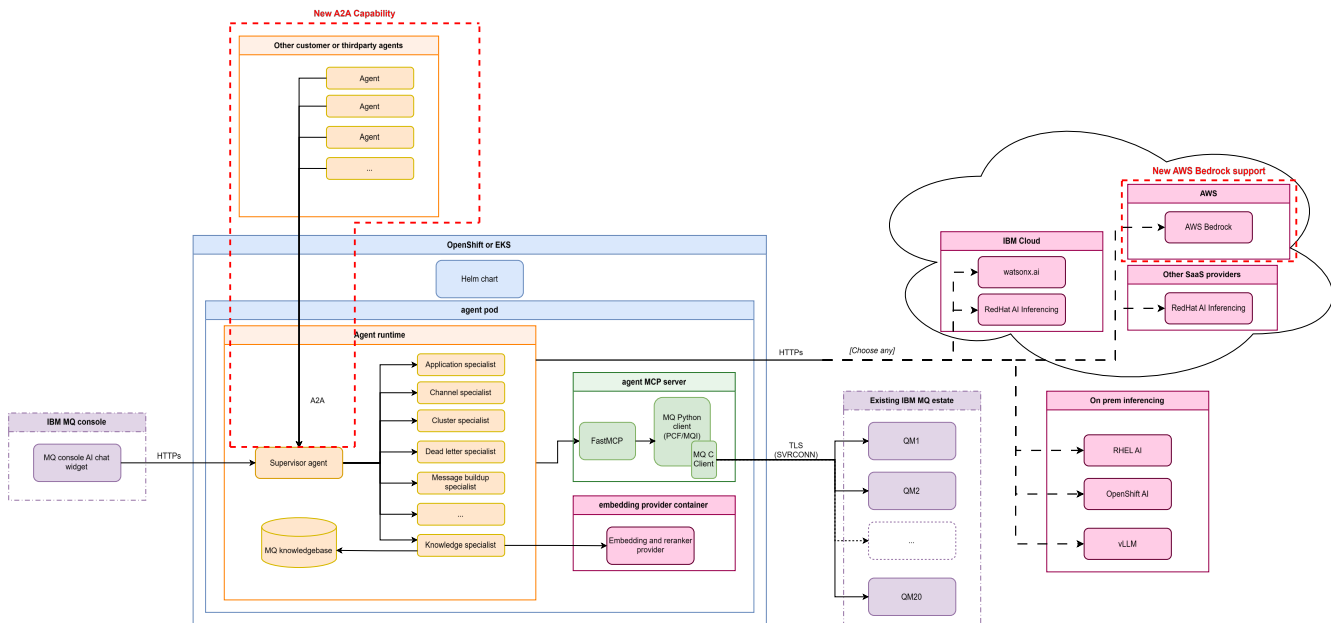


Figure 4, IBM MQ Agent A2A Integration and AWS Bedrock support

1.2.2.1 Summary of general MQ agent capabilities

The IBM MQ agent capabilities can be broadly categorised as three themes:

- **Teach**
Explain MQ concepts, objects and operational practices citing authoritative source material.
- **Show**
Work with connected queue managers to query, filter and present MQ objects, their configuration and real-time status.
- **Diagnose**
Provide deep insights into configuration and run-time state to support users in identifying issues and proposing remedial actions

The agent uses conversation context and conversation history to select which of its capabilities can help you analyze your IBM MQ configuration and diagnose problems. Here are some examples of the type of question and problem determination that the IBM MQ Agent can help you with:

Questions about IBM MQ objects and status

- Report cluster memberships of a queue manager
- Report IBM MQ versions and platforms in your queue manager estate
- Show the queues that are available on a queue manager or set of queue managers
- Identify queues that have a current depth of more than 100 messages (for example)
- Identify queues on queue managers where messages haven't been processed for more than 30 minutes (for example)
- Identify queues with no connected consuming applications
- Report queues with a specified attribute, for example list shared queues on a queue manager
- Show channels that are in a retry state
- Show all running listeners on a queue manager or set of queue managers
- Identify where an application is connected
- Explain queue attributes and their purpose

Problem determination

- Diagnose why a channel is in a stopped state
- Understand why messages were delivered to a dead letter queue
- Search error logs to help diagnose problems that are related to channels

- Understand which ports listeners are running on
- Identify and diagnose general message build-up problems
- Diagnose slow message flow over a network
- Understand why messages are building up

Intelligent insights and next step planning

- Explain an IBM MQ concept
- Reason about configuration issues
- Predict future outcomes based on real-time status
- Help you plan your next steps to resolve a problem

Examples questions

Here are some examples of general questions and instructions that you might ask the IBM MQ Agent:

1. What queue managers can you see?
2. What version are my queue managers?
3. Do I have message build up?
4. What is the problem with *<queue name>* on *<queue manager name>*?
5. Diagnose *<queue name>* on *<queue manager name>*
6. Why are messages building up on *<queue name>* on *<queue manager name>*?
7. Is there a problem with the dead letter queue on *<queue manager name>*?
8. Explain the UNCOM attribute of queue status.

1.2.3 Working with the agent

Follow these guidelines when you ask questions:

1. If you ask about an object, provide the object type as well.
2. If you want the output to include system queues, ask for them explicitly; the IBM MQ Agent omits them by default. For example: "List all my queues on queue manager qm1 including system queues".
3. Focus on one conversation at a time, otherwise the IBM MQ Agent might ask you clarification questions.

4. If a response to your question results in a large volume of data, the large language model (LLM) might summarize the information. If you suspect that a response is missing information, repeat your question but ask for a full answer.

Note: The IBM MQ Agent cannot run commands and is not intended as a general purpose AI chat interface.

The IBM MQ Agent provides regular status updates as it processes your questions by working through its planning, information gathering, and reasoning steps. For example, you might see status messages similar to the following:

- Analyzing query
- Planning
- Working [step 1 of 1] -> Checking queue status

If you want to restart a conversation, click the **Restart Conversation** button.

You might want to restart a conversation when you start analyzing a new problem or if the output starts to deviate from your expectations. Deviation can occur with long conversations because context can leak across questions, particularly when you are working across different queue managers and objects with duplicate names. Similarly, in longer running conversations the chat assistant might shift to focus on more recent context.

Optional: If you want to save your conversation, click the three dot menu icon at the top of the chat window then click **Save conversation**.

You might want to save conversations for use in internal support tickets, for example, or for deeper analysis of problems.

The conversation history is saved as a JSON file to your local machine, according to your web browser preferences. Here is an example of a saved conversation:

1.3 Conventions

All text that you need to copy, and paste are presented in the following format:

```
an-example-lab-command
```

All URLs that you need to copy, and paste are presented in the following format:

```
https://an-example-lab-url
```

2 Timing

The key objectives of this lab have been designed to fit a 90 minute session with allowances for initial setup time. Should we encounter a technical problem, or if you would like to repeat an element of the lab, information has been provided in [Section 3](#). on how to access an environment outside of this MQ Summit session. The approximate time allocation is:

Part	Activity	Time
------	----------	------

A	Introduction and hands-on examples	~25 minutes
B	Scenario 1 (either option)	~25 minutes
C	Scenario 2 (either option)	~25 minutes
-	Q&A / wrap-up	~15 minutes

3 Didn't get to complete the lab?

If you run out of time during the session, or would like to revisit the lab content after The MQ Summit, please reach out to the lab instructor at coppen@uk.ibm.com. Information on accessing IBM MQ AI Agents outside of the MQ Summit lab environment will be made available to registered attendees.

This [blog](#) provides a deep dive into the IBM MQ agent architecture and capabilities. You can also learn more about the IBM MQ agent capabilities and deployment in the [IBM MQ documentation](#). For general IBM MQ learning resources, the following resources might be useful:

IBM MQ developer learning: <https://ibm.biz/learn-mq>
IBM MQ developer patterns: <https://ibm.biz/mq-dev-patterns>
IBM MQ YouTube playlist: <https://ibm.biz/mq-playlist>

4 About this hands-on lab

This lab has three parts:

Part	Content
A	Introduction and hands-on examples — build familiarity with the AI Agents interface and learn how to ask effective questions
B	Scenario 1 — triage a queue manager with multiple queue-related problems
C	Scenario 2 — triage an inter-queue-manager connectivity and performance problem

For Parts B and C, you have a choice of two options depending on your experience level:

- **Option 1** — Support ticket track (advanced): You receive a realistic support ticket describing the reported symptoms. You use the AI Agents to investigate from first principles and identify the root cause. No hints are provided — this mirrors real operational practice.
- **Option 2** — Guided prompts track (assisted): You follow a structured sequence of prompts that lead you step by step to the root cause. Use this option if you are newer to IBM MQ, if you prefer a more structured approach, or if you get stuck on Option 1.

Both options lead to the same root cause findings. You can switch between them at any time.

By the end of this lab you will be able to:

- Navigate the IBM MQ AI Agents interface

- Use natural language to query queue manager state and metrics
- Identify queue-related operational problems (inhibited queues, uncommitted messages, dead-letter queue saturation, insufficient message length settings)
- Identify inter-queue-manager channel performance problems
- Propose and articulate remediation steps

4.1 Agent capabilities

The key agent capabilities you will use in this lab are:

Key capabilities used in this lab:

Capability	Description
Queue depth analysis	Query current depth, maximum depth, and percentage capacity for any queue
Queue attribute inspection	Retrieve queue properties such as GET, PUT, MAXMSGL, MONQ settings
Application connection analysis	Identify which applications are connected and whether they have open input or output handles
Uncommitted message detection	Identify queues with a high proportion of uncommitted (in-flight) puts
Channel status monitoring	Query sender, receiver, and server-connection channel states and statistics
Channel performance analysis	Examine network round-trip times and channel throughput metrics
Dead-letter queue analysis	Inspect DLQ contents and correlate messages with their intended destination queues
Remediation guidance	Propose the specific MQSC commands or operational steps needed to resolve a problem

4.2 Queue managers

For reference, you will be working with the following queue managers.

Queue Manager	Purpose
QMS1	Single queue manager used in Scenario 1 (Part B)
QMS2A	Sending queue manager used in Scenario 2 (Part C)
QMS2B	Receiving queue manager used in Scenario 2 (Part C)

4.3 Lab environment reference

WARNING: This reference provides debug clues. If you have selected Option 1 skip this Section unless you are looking for a hint. [Click here to jump to the exercises in Section 5](#)

The lab environment you are working with has been pre-configured with all queue managers, queues, channels, and simulated applications needed to complete the tasks. The environment is designed to present realistic operational problems for you to investigate.

Scenario 1 environment (QMS1):

Object	Type	Notes
ACME.APPDATA.QUEUE	Local Queue	85% full, get-inhibited, no consumers
ACME.COMMERCE.QUEUE	Local Queue	90% full, get-inhibited, 80% uncommitted puts
ACME.NOTIFICATION.QUEUE	Local Queue	Empty, MAXMSGL set too small
SYSTEM.DEAD.QUEUE	Local queue	90% full, messages destined for ACME.NOTIFICATION.QUEUE
AcmeAltProducer	Application	Putting messages to ACME.COMMERCE.QUEUE (not committing)
AcmeConsumer	Application	Connected to ACME.COMMERCE.QUEUE

Scenario 2 environment (QMS2A → QMS2B):

Object	Type	Notes
PRODAPP1.INPUT (QMS2A)	Alias queue	Alias to PRODAPP1.REMOTE
PRODAPP1.REMOTE (QMS2A)	Remote queue	Points to PRODAPP1.INPUT on QMS2B
QMS2B (QMS2A)	Transmission queue	Holds messages for delivery to QMS2B
QMS2A.TO.QMS2B (QMS2A)	SDR channel	Sending channel with simulated slow network link
QMS2A.TO.QMS2B (QMS2A)	SDR channel	Sending channel with simulated slow network link
PRODAPP1.INPUT (QMS2B)	Local queue	Destination queue
QMS2A.TO.QMS2B (QMS2B)	RCVR channel	Receiving end of the channel

5 Part A – Introduction and hands-on examples

This part familiarises you with the IBM MQ AI Agents interface before you tackle the scenarios. You will run a series of hands-on examples against the pre-configured queue managers and verify the results using the IBM MQ Console.

5.1 Connecting to the AI Agents interface

1. Open a web browser on your student workstation
2. Navigate to the MQ Console at <https://158.175.71.201:31414/ibmmq/console/> or <https://158.177.74.129:31811/ibmmq/console/>
3. Since the console is using a self-signed certificate as part of the default configuration, you will see the warning message “Your connection is not private.”
 - Click Advanced
 - Click Proceed to localhost (unsafe)
 - The IBM MQ Console will now continue to load

4. You should now see the console load in the browser
If prompted, login with these credentials
User: `mqreader`
Password: `mqreader`
5. Click the AI icon (Figure 5) to open the AI agent chat interface

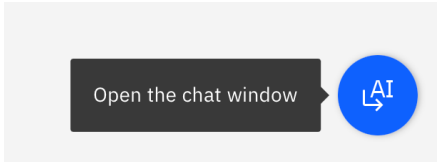


Figure 5, AI chat window icon

6. The chat interface will open in the right pane of the console session as per (Figure 6)

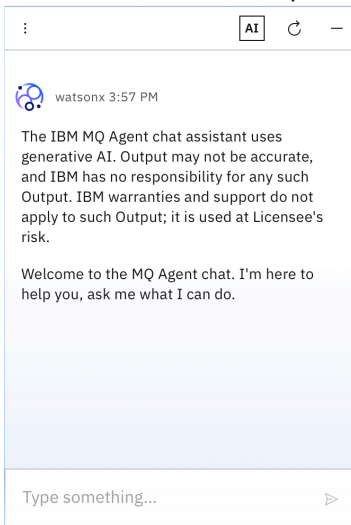


Figure 6, IBM MQ Agent chat interface

7. Confirm that the interface is connected correctly by asking the agent to list the available queue managers. Type the following command and press enter.

`What queue managers can you see?`

You should observe the agents process the request and return **QMS1**, **QMS2A** and **QMS2B** as a list of queue managers.

8. If no queue managers are listed ask the lab instructor for assistance.

5.2 Hands-on example 1 — Exploring queue manager health

In this example you will ask the AI Agents to give you a high-level health summary of QMS1 and then drill down into the results.

1. In the AI Agents interface, type the following prompt and press Enter:

`Give me a health summary for queue manager QMS1`

2. Review the response. The agent will query the queue manager and return a summary that includes queue depths, channel states, and any items of concern it has identified.
3. Note any queues the agent flags as having elevated depth. You will investigate these in Part B.
4. Now ask the agent to list all queues on QMS1 that are above 50% capacity:

List all queues on QMS1 that are more than 50% full

5. Compare the results with the IBM MQ Console. In the Console, navigate to **Queue Managers → QMS1 → Queues** and verify that the queues reported by the agent match what you see.
Tip: The AI Agents are reading live data from the queue manager — the same data the console is showing you. This gives you confidence that the agents are working correctly.
6. Finally, ask the agent to explain what the current queue depth percentage means in operational terms:

What are the risks if a queue reaches 100% capacity on QMS1?

This demonstrates that the agents can combine general MQ knowledge with the live data from your environment.

5.3 Hands-on example 2 – Investigating channel status

In this example you will ask the AI Agents about the state of channels on QMS2A.

1. In the AI Agents interface, type:

What is the status of all channels on QMS2A?

2. Review the response. You should see the QMS2A.TO.QMS2B sender channel listed along with its current state (running, retrying, stopped, etc.).
3. Ask the agent for more detail on the sender channel:

Give me detailed statistics for the QMS2A.TO.QMS2B channel

4. Note the network round-trip time (network time) reported for the channel. You will return to this in Part C.
5. In the IBM MQ Console, navigate to **Manage → Remote queue managers → remote-QMS2A → Channels** and locate QMS2A.TO.QMS2B. Compare the statistics with what the agent reported.

5.4 Hands-on example 3 — Correlating applications and queues

In this example you will ask the AI Agents to tell you which applications are connected to QMS1 and what they are doing.

1. In the AI Agents interface, type:

Which applications are currently connected to QMS1?

2. Review the list of connected applications and the queues they have open.

3. Ask the agent to identify which applications have open output handles (i.e. are currently putting messages):

Which applications on QMS1 are currently putting messages to queues?

4. Then ask whether any of those applications have uncommitted puts:

Are any applications on QMS1 putting messages without committing them?

5. Note the application name that the agent identifies. You will need this in Part B.

Tip: Uncommitted puts are messages that have been written to a queue inside a transaction that has not yet been committed. They occupy queue space and count toward queue depth, but they cannot be consumed by a receiving application until the transaction commits. If an application hangs mid-transaction, uncommitted puts will accumulate and can fill a queue.

6 Part B – Scenario 1: Queue manager alert triage

Scenario background

It is Monday morning at AcmeCorp. You are a member of the MQ operations team. Over the weekend, a scheduled maintenance window was performed on **QMS1**. The engineers have since gone home.

Your monitoring system has fired alerts indicating that queues are approaching capacity on **QMS1**, but — due to a misconfiguration made during the maintenance window — the alerts are no longer reporting the specific queue names. You know queues are filling up, but not which ones or why.

Your task is to investigate **QMS1**, identify all the problems, and produce a root cause analysis with a proposed remediation plan.

There are three separate root causes to find. Each requires a different fix.

6.1 Option 1 – Support ticket (advanced)

Use this option if you want to investigate from first principles, as you would in a real operational scenario. No hints are given. Skip to [Section 6.2](#) if you prefer the guided approach.

SUPPORT TICKET – INC0042781

Field	Value
Priority	P2 — High
Raised by	Automated monitoring system
Assigned to	MQ Operations

Field	Value
Time raised	Monday 08:14
Status	Open

Summary: Queue capacity alerts firing on QMS1 — queue names not reported

Description:

Capacity threshold alerts have been triggered on queue manager QMS1. The alerts indicate that one or more queues are approaching or have reached their configured maximum depth. However, following the maintenance window performed over the weekend, the alert configuration was inadvertently modified, and the affected queue names are no longer included in the alert notifications.

The application teams have not yet reported any failures, but this is expected to escalate if the queues fill completely and messages begin to back up or be rejected.

Action required:

- Identify all queues on QMS1 that are at elevated depth
- Determine the root cause of the elevated depth for each affected queue
- Propose a remediation plan that includes the specific operational steps or MQSC commands required to resolve each issue
- The remediation plan must be suitable for submission as a production change request

Use the IBM MQ AI Agents to investigate QMS1 and answer the ticket. When you believe you have found all three root causes, check your findings against the answer key in [Section 10](#).

Record your findings here:

#	Queue affected	Root cause	Proposed remediation
1			
2			
3			

6.2 Option 2 — Guided prompts (assisted)

Use this option if you are newer to IBM MQ, prefer a structured approach, or have got stuck on Option 1. Each prompt builds on the previous one to lead you to all three root causes.

Work through the following prompts in order. After each one, read the agent's response carefully before moving to the next prompt.

Step 1 — Get an overview of queue depths

Enter the following prompt:

List all queues on QMS1 that are more than 50% full, including their current depth, maximum depth, and percentage full

What to look for: The agent should report several queues at elevated depth, including `ACME.APPDATA.QUEUE`, `ACME.COMMERCE.QUEUE`, and `SYSTEM.DEAD.QUEUE`. Note which queues appear and their approximate capacity percentages before continuing.

Step 2 – Investigate the first queue (`ACME.APPDATA.QUEUE`)

Enter the following prompt:

Why is `ACME.APPDATA.QUEUE` filling up on QMS1? Check its GET and PUT settings and whether there are any consumers connected

What to look for: The agent should report that `ACME.APPDATA.QUEUE` has `GET (INHIBITED)` set, meaning no application can consume messages from it. It should also confirm there are no active consumers (no open input handles). The queue is filling because messages can be put to it but nothing can take them off.

Root cause 1: `ACME.APPDATA.QUEUE` is get-inhibited.

Proposed fix – ask the agent:

What MQSC command would I use to re-enable gets on `ACME.APPDATA.QUEUE` on QMS1?

Note the command the agent provides. It should be:

```
ALTER QLOCAL (ACME.APPDATA.QUEUE) GET (ENABLED)
```

Step 3 – Investigate the second queue (`ACME.COMMERCE.QUEUE`)

Enter the following prompt:

Why is `ACME.COMMERCE.QUEUE` filling up on QMS1? Check consumer and producer connections, and whether there are any uncommitted messages

What to look for: The agent should report that `ACME.COMMERCE.QUEUE` also has `GET (INHIBITED)` set. It should also identify that approximately 80% of the messages currently on the queue are **uncommitted** — they have been put by an application, but the transaction has not been committed. This is unusual and indicates a misbehaving producer.

Ask the agent to identify the offending application:

Which application on QMS1 is putting uncommitted messages to `ACME.COMMERCE.QUEUE`?

What to look for: The agent should identify `AcmeAltProducer` as the application that is putting messages to `ACME.COMMERCE.QUEUE` without committing them.

Root cause 2: `AcmeAltProducer` is putting messages to `ACME.COMMERCE.QUEUE` without committing its transactions, causing uncommitted messages to accumulate.

Proposed fix – ask the agent:

What steps should I take to address the uncommitted puts from `AcmeAltProducer` on `ACME.COMMERCE.QUEUE` on `QMS1`?

Note the agent's recommendation. It should include escalating to the `AcmeAltProducer` application team to investigate the uncommitted transaction behaviour and potentially disconnecting the misbehaving application instance.

Step 4 – Investigate the dead-letter queue

Enter the following prompt:

Why is `SYSTEM.DEAD.QUEUE` filling up on `QMS1`? What messages are on it and where were they originally destined?

What to look for: The agent should report that the dead-letter queue contains messages that were originally destined for `ACME.NOTIFICATION.QUEUE`. Messages are ending up on the DLQ because they cannot be delivered to their intended destination.

Ask the agent to investigate why:

Why can't messages be delivered to `ACME.NOTIFICATION.QUEUE` on `QMS1`? Check its `MAXMSGL` setting

What to look for: The agent should identify that `ACME.NOTIFICATION.QUEUE` has an unusually small `MAXMSGL` (maximum message length) value – too small to accept the messages being sent to it. When the queue manager attempts to put a message that exceeds `MAXMSGL`, it routes it to the dead-letter queue instead.

Root cause 3: `ACME.NOTIFICATION.QUEUE` has an insufficient `MAXMSGL` setting, causing messages to be redirected to the dead-letter queue.

Proposed fix – ask the agent:

What MQSC command would I use to increase the `MAXMSGL` on `ACME.NOTIFICATION.QUEUE` on `QMS1` to 4194304?

Note the command the agent provides. It should be:

```
ALTER QLOCAL (ACME.NOTIFICATION.QUEUE) MAXMSGL (4194304)
```

Step 5 – Summarise your findings

Enter the following prompt to ask the agent to compile a summary suitable for a change request:

Produce a summary of all the problems found on QMS1 and the MQSC commands needed to fix them, formatted as a production change request

Review the output. It should cover all three root causes and the associated remediation commands.

Completed findings – Scenario 1:

#	Queue affected	Root cause	Remediation command / action
1	ACME.APPDATA.QUEUE	GET (INHIBITED) – queue is get-inhibited, no consumers can read from it	ALTER QLOCAL (ACME.APPDATA.QUEUE) GET (ENABLED)
2	ACME.COMMERCE.QUEUE	AcmeAltProducer is putting messages without committing transactions – 80% of depth is uncommitted	Engage AcmeAltProducer application team; disconnect misbehaving instance
3	SYSTEM.DEAD.QUEUE (root cause on ACME.NOTIFICATION.QUEUE)	MAXMSGL on ACME.NOTIFICATION.QUEUE is too small – messages are routed to DLQ	ALTER QLOCAL (ACME.NOTIFICATION.QUEUE) MAXMSGL (4194304)

7 Part C – Scenario 2: Application performance triage

Scenario background

It is 3am at AcmeCorp and you have been paged out. A new severity-1 ticket has been raised against the MQ operations team by an application team. You need to provide a root cause analysis.

7.1 Option 1 – Support ticket (advanced)

Use this option if you want to investigate from first principles. Skip to [Section 7.2](#) if you prefer the guided approach.

SUPPORT TICKET – INC0043102

Field	Value
Priority	P1 – Critical
Raised by	Application team (prodApp1)
Assigned to	MQ Operations
Time raised	03:07
Status	Open

Summary: prodApp1 running slowly — suspected MQ issue

Description:

Our Instana dashboards are showing that prodApp1 is running significantly slower than baseline. Response times have degraded by approximately 300% over the past two hours.

We have checked the following and found no issues:

- Application error logs — no errors or warnings
- Network team — confirmed network is healthy

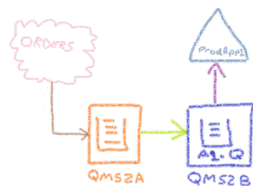
The problem must be on the MQ side.

I have attached a diagram of the expected message flow for prodApp1:

prodApp1 → PRODAPP1.INPUT (QMS2A alias) → PRODAPP1.REMOTE (QMS2A) → [channel] → PRODAPP1.INPUT (QMS2B)

Please investigate and provide a root cause analysis.

Attached: Network diagram showing expected queue manager topology



Use the IBM MQ AI Agents to investigate the message flow and identify the root cause. When you believe you have found it, check your findings against the answer key in [Section 10](#).

Record your findings here:

Field	Your finding
Root cause:	
Affected component:	
Evidence from AI Agents:	
Proposed remediation:	

7.2 Option 2 — Guided prompts (assisted)

Use this option if you prefer a structured approach or have got stuck on Option 1.

Work through the following prompts in order.

Step 1 – Understand the message flow

Enter the following prompt:

Explain the message flow for a message put to PRODAPP1.INPUT on QMS2A. What queues and channels does it traverse to reach its destination?

What to look for: The agent should describe the full path:

PRODAPP1.INPUT (alias) → PRODAPP1.REMOTE (remote queue definition)
→ QMS2B (transmission queue) → QMS2A.TO.QMS2B (SDR channel)
→ PRODAPP1.INPUT on QMS2B.

This confirms you understand the topology before investigating.

Step 2 – Check queue depths along the flow

Enter the following prompt:

Check the depth of the transmission queue QMS2B on QMS2A. Are messages accumulating there?

What to look for: If messages are accumulating in the transmission queue, it indicates that they are not being forwarded to **QMS2B** at the expected rate — pointing to a problem with the **QMS2A.TO.QMS2B** channel.

Step 3 – Examine the channel

Enter the following prompt:

Give me detailed performance statistics for the QMS2A.TO.QMS2B channel on QMS2A, including network time and any performance metrics

What to look for: The agent should report the channel's current statistics. Pay particular attention to the **network time** (round-trip time) metric. A high network time indicates that the link between the two queue managers is slow — messages are waiting a long time for acknowledgements from **QMS2B**.

Step 4 – Confirm the root cause

Enter the following prompt:

Is the network latency on the QMS2A.TO.QMS2B channel abnormally high?

What to look for: The agent should confirm that the network round-trip time on **QMS2A.TO.QMS2B** is significantly elevated — consistent with a slow network link between the two queue managers. This explains why prodApp1 is running slowly: the application is waiting for messages to traverse the inter-QM channel, and that channel is experiencing high latency.

Root cause: Slow network link on the `QMS2A.TO.QMS2B` sender channel, causing high message delivery latency between `QMS2A` and `QMS2B` and consequent slowness in `prodApp1`.

Step 5 – Produce the root cause analysis

Enter the following prompt:

```
Produce a root cause analysis for a slow prodApp1 caused by the QMS2A.TO.QMS2B channel. Include the evidence, the impact, and recommended next steps
```

Review the agent's output. It should form the basis of your response to ticket INC0043102.

Completed findings – Scenario 2:

Field	Finding
Root cause	Excessive network latency on the <code>QMS2A.TO.QMS2B</code> SDR channel
Affected component	<code>QMS2A.TO.QMS2B</code> sender channel / network link between <code>QMS2A</code> and <code>QMS2B</code>
Evidence	Elevated network round-trip time on the channel; messages accumulating in the <code>QMS2B</code> transmission queue on <code>QMS2A</code>
Impact	Messages destined for <code>PRODAPP1.INPUT</code> on <code>QMS2B</code> are delayed in transit, causing <code>prodApp1</code> to run slowly
Recommended next steps	Engage network team with specific channel performance data; investigate network path between the two queue manager hosts

8 Feedback section

We hope you enjoyed this lab and found the IBM MQ AI Agents useful. We would very much appreciate your feedback – it directly influences how this technology develops.

If you have questions or would like to continue the conversation after the lab, please contact the instructor:

- **Email:** coppen@uk.ibm.com
- **LinkedIn:** www.linkedin.com/in/richard-coppen

9 Getting help and troubleshooting

Reach out to the lab instructor

The instructors will be available for help and support throughout the lab and available after the session for general Q&A. If you would like to ask a question after the labs been completed, the instructors can be contacted via email at coppen@uk.ibm.com.

Common troubleshooting tips

The AI Agents interface is not responding

1. Refresh the browser tab.
2. Restart the chat session.
3. If the problem persists, check that the AI Agents service is running. Ask the lab instructor for assistance.

The agent returns an error when querying a queue manager

1. Confirm you are using the exact queue manager name (names are case-sensitive): **QMS1**, **QMS2A**, **QMS2B**.
2. Try rephrasing your prompt or breaking your prompt down into smaller prompts that build on each other.

The IBM MQ Console will not load

1. Ensure you have accepted the self-signed certificate warning in your browser.
2. Navigate directly to: <https://158.175.71.201:31414/ibmmq/console/>
3. Log in with username `mqreader` and password `mqreader`.

Getting help

If you experience a problem that is not covered above, please raise your hand and ask the lab instructor. For post-session support, email coppen@uk.ibm.com

10 Lab reference materials

Answer key – Scenario 1 (QMS1)

The three root causes for the queue capacity alerts on QMS1 are:

1. **ACME.APPDATA.QUEUE is get-inhibited (GET (INHIBITED)).**
No application can consume messages, so the queue fills up even with no active producers.
Fix: `ALTER QLOCAL (ACME.APPDATA.QUEUE) GET (ENABLED).`
2. **AcmeAltProducer is putting messages to ACME.COMMERCE.QUEUE without committing transactions.**
Approximately 80% of the queue depth consists of uncommitted messages from this application.
Fix: engage the `AcmeAltProducer` application team to investigate and correct the uncommitted transaction behaviour; disconnect the misbehaving application instance while the fix is applied.
3. **ACME.NOTIFICATION.QUEUE has an insufficient MAXMSGL setting.**
Messages destined for `ACME.NOTIFICATION.QUEUE` exceed the configured maximum message length and are therefore routed to `SYSTEM.DEAD.QUEUE` instead.

Fix: `ALTER QLOCAL (ACME.NOTIFICATION.QUEUE) MAXMSGL (4194304)`. After fixing, the DLQ messages can be reprocessed.

Note: `ACME.COMMERCE.QUEUE` is also get-inhibited. Students who identify this additional attribute of that queue as a contributing factor are correct — both `GET(INHIBITED)` and the uncommitted puts are contributing to a high queue depth.

Answer key – Scenario 2 (QMS2A / QMS2B)

The root cause of `prodApp1` running slowly is:

- **Slow network link on the QMS2A.TO.QMS2B sender channel.**

The network round-trip time (network time) for this channel is significantly elevated.

Because `prodApp1` puts messages to `PRODAPP1.INPUT` on `QMS2A`, which are forwarded via the `QMS2A.TO.QMS2B` channel to `PRODAPP1.INPUT` on `QMS2B`, high channel latency directly impacts `prodApp1`'s end-to-end response time. The network and application teams reported no application or network issues — however, the slow link exists specifically between the two queue manager hosts and was not captured by general network monitoring.

Fix: engage the network team with the specific channel performance data (host names, ports, observed round-trip time) to investigate and remediate the slow path.

Further reading and resources

Resource	URL
IBM MQ documentation	https://ibm.biz/mq-docs
IBM MQ developer learning	https://ibm.biz/learn-mq
IBM MQ developer patterns	https://ibm.biz/mq-dev-patterns
IBM MQ YouTube playlist	https://ibm.biz/mq-playlist
IBM MQ advanced topics playlist	https://ibm.biz/mq-playlist-advanced
IBM MQ Developer Essentials badge	https://ibm.biz/mq-badge